Memory Management in Linux

Self-Learning Practice Report – Operating Systems (B24CS2T04)

Topic 5: Memory Management in Linux
Sub-topics:

a) Virtual Memory in Linux
b) Demand Paging – Demonstration with a Memory-Intensive Program
c) Comparing Page Faults using vmstat

Course: B24CS2T04 – Operating Systems
Course Outcome Addressed: CO 4 – Understand memory management techniques and their role in optimizing system performance (Cognitive Level: Apply)

Table of Contents

1. Introduction
2. Theoretical Background
   - 2.1 Address Spaces & Memory Abstraction
   - 2.2 Virtual Memory in Linux
   - 2.3 Demand Paging
   - 2.4 Page Replacement Algorithms
   - 2.5 Thrashing
3. Implementation Details with OS-Level Explanation
   - 3.1 Program Design
   - 3.2 Kernel-Level Walkthrough
   - 3.3 vmstat Monitoring Setup
4. Experimental Results
   - 4.1 Test Environment
   - 4.2 Phase-wise Page Fault Observations
   - 4.3 Comparison Across Access Patterns
   - 4.4 vmstat Output Analysis

5. SDG Mapping
6. Conclusion
7. References

---

1. Introduction

Memory management is one of the most critical subsystems of any operating system. It determines how physical RAM is shared among processes, how the illusion of a large contiguous address space is maintained, and how the system degrades gracefully when physical memory is exhausted.

Linux, being a modern monolithic kernel, implements a sophisticated memory-management subsystem that includes:

- **Virtual memory** – each process gets its own isolated 64-bit (or 32-bit) address space.
- **Demand paging** – physical frames are assigned only when a page is actually accessed, not when it is allocated.
- **Page replacement** – when physical memory is full, the kernel evicts less-used pages to swap using algorithms approximating LRU.

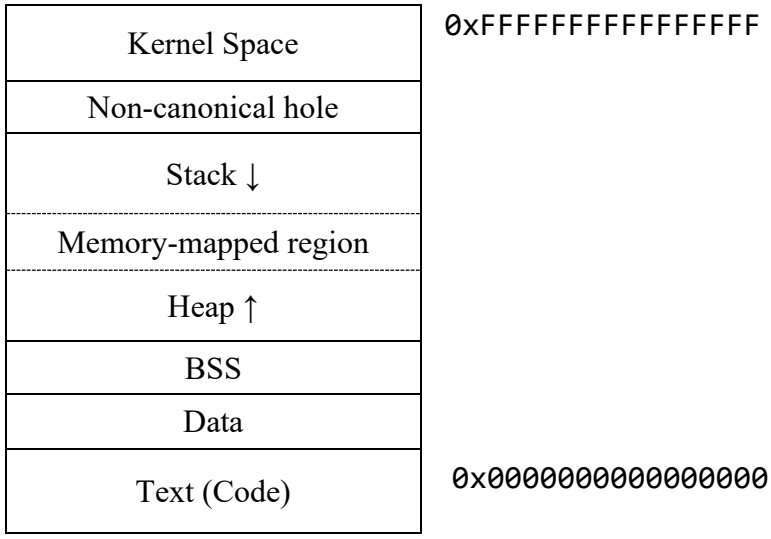
This report explores these three pillars both theoretically and experimentally. We write a C program that allocates a large block of memory and touches pages in different patterns (sequential, random, stride), measuring minor and major page faults using /proc/self/stat. We then corroborate the results with vmstat, a standard Linux monitoring utility.

---

2. Theoretical Background

2.1 Address Spaces & Memory Abstraction

Every Linux process operates in its own **virtual address space**. On a 64-bit x86 system, the user-space range is 0 – 0x00007FFFFFFFFFFF (128 TB). The kernel occupies the upper canonical half. This separation is enforced by hardware (page tables + MMU) and ensures that one process cannot corrupt another.



2.2 Virtual Memory in Linux

Virtual memory decouples the address a program uses from the physical RAM location that actually holds the data. The mapping is maintained in **page tables** — hierarchical structures (4-level on x86-64: PGD → PUD → PMD → PTE) that the Memory Management Unit (MMU) walks on every memory access.

Key concepts:

Concept	Description
Page	A fixed-size block of virtual memory (typically 4 KB on x86).
Frame	A fixed-size block of physical RAM — same size as a page.

Concept	Description
Page Table Entry (PTE)	Maps one virtual page to one physical frame; stores permission bits, dirty bit, accessed bit, present bit.
Translation Lookaside Buffer (TLB)	CPU cache of recent PTE translations. TLB miss → expensive page-table walk.
Page Fault	CPU exception raised when the present bit is 0. The kernel then decides whether to (a) allocate a frame (minor fault), (b) load from disk/swap (major fault), or (c) send SIGSEGV (invalid access).

How the Linux kernel manages virtual memory:

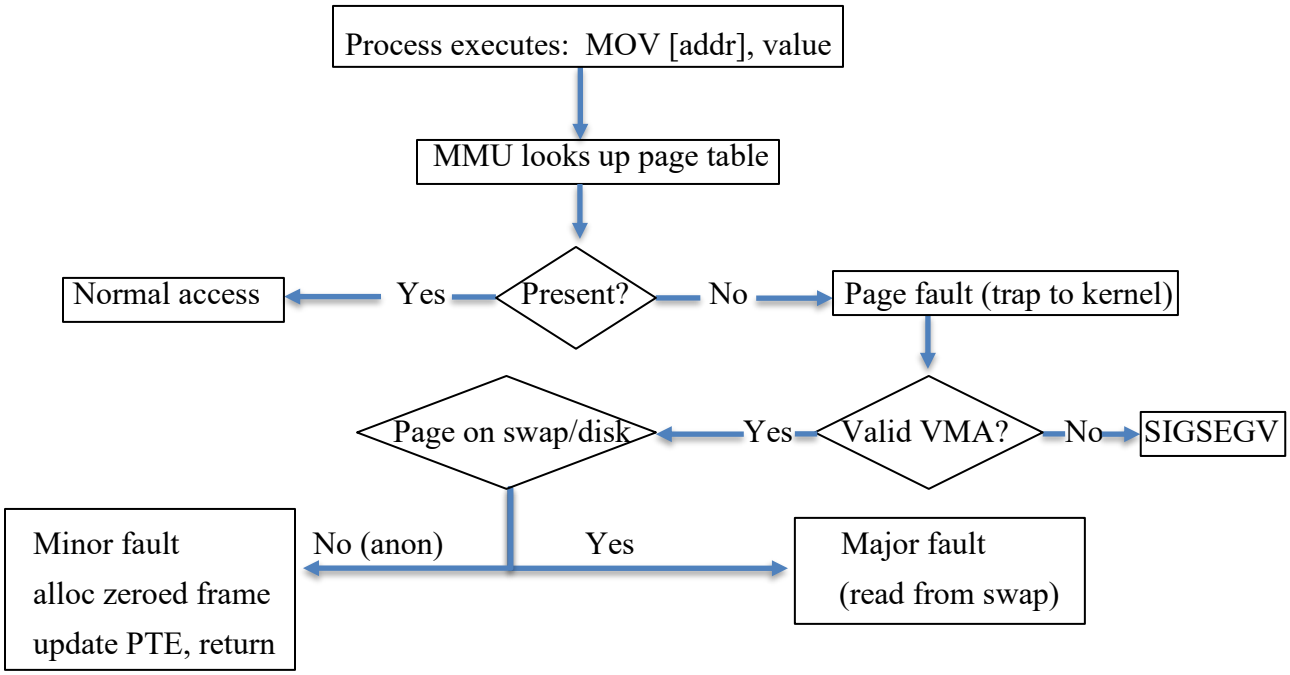
1. struct mm_struct — per-process memory descriptor; holds the page-table root (pgd) and the list of Virtual Memory Areas (VMAs).
2. struct vm_area_struct — describes one contiguous region (text, heap, mmap, stack) with its start/end addresses, permissions, and backing file (if any).
3. When malloc() is called, glibc invokes brk() or mmap(). The kernel creates/extends a VMA but does **not** allocate physical frames — this is *lazy allocation*.

2.3 Demand Paging

Demand paging is the mechanism by which the kernel defers physical memory allocation until the exact moment a page is first accessed.

- **Minor fault:** The kernel allocates a zeroed physical frame, installs the PTE, and the instruction restarts transparently. No disk I/O is needed.
- **Major fault:** The page's data must be read from swap (or a memory-mapped file); this involves disk I/O and is orders of magnitude slower.

Workflow:



2.4 Page Replacement Algorithms

When physical memory is exhausted, the kernel must evict pages. Linux uses a **two-list LRU approximation** — also called a **modified 2Q** algorithm (OSTEP §23.2) — consisting of an **active list** and an **inactive list**, managed by the kswapd daemon. The active list is kept at approximately **two-thirds** of the total page cache; when a page on the inactive list is re-accessed it is promoted to the active list, and kswapd periodically demotes pages from the active list to the inactive list to maintain the ratio:

Algorithm	Description	Used in Linux?
FIFO	Evict the oldest page. Simple but suffers from Belady's anomaly.	No
Optimal	Evict the page not used for the longest future time. Theoretical ideal; not implementable.	No

Algorithm	Description	Used in Linux?
LRU	Evict the least-recently-used page. Good but expensive to implement exactly (requires timestamp or stack on every access).	Approximated
LRU-Approximation (Clock / Second Chance)	Uses the "accessed" bit in the PTE. Pages get a second chance before eviction.	Yes (via active/inactive lists)

2.5 Thrashing

When the **working set** of a process exceeds available physical memory, the system continuously pages in and out — every page brought in evicts another page that will soon be needed. This is called **thrashing**. Symptoms: CPU utilisation drops, disk I/O saturates, vmstat shows high si/so (swap in/swap out).

---

3. Implementation Details with OS-Level Explanation

3.1 Program Design (demand_paging_demo.c)

The program performs three phases:

Phase	Action	Expected Faults
Phase 1	malloc(N) – reserves virtual addresses	~0 (no physical allocation)
Phase 2	Touch every page using one of three access patterns	~N/4096 minor faults
Phase 3	Re-touch all pages sequentially	~0 (pages already resident)

Access patterns:

- **Sequential (seq):** Pages are touched in order from address 0 to N. Good spatial locality; prefetch-friendly.
- **Random (random):** Pages are touched in pseudo-random order using a Fisher-Yates (Knuth) shuffle, ensuring every page is visited exactly once. Poor TLB locality; more TLB misses though total page fault count is identical to sequential.
- **Stride-64 (stride):** Every 64th page is touched, then we loop back. Moderate locality; demonstrates the effect of access granularity.

The program reads /proc/self/stat fields 10 (minflt) and 12 (majflt) to measure page-fault counts at each phase boundary.

3.2 Kernel-Level Walkthrough

When demand_paging_demo calls malloc(512 * 1024 * 1024):

1. **glibc** decides this is a large allocation and issues mmap(NULL, 512MB, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0).
2. The **kernel** creates a new vm_area_struct under the process's mm_struct. The VMA covers 512 MB of virtual addresses. No physical frames are allocated; the page-table entries are either absent or marked not-present.
3. When the program writes mem[0] = 'A':
    - The MMU finds no valid PTE → triggers a **page fault** (interrupt 14 on x86).
    - The kernel's do_page_fault() → find_vma() (locates the VMA containing the faulting address) → handle_mm_fault() → handle_pte_fault() is called.
    - Since this is an anonymous private mapping and the PTE is empty, do_anonymous_page() allocates a zeroed 4 KB frame via alloc_page() (which uses the buddy allocator), installs a new PTE, and returns.

- The faulting instruction is re-executed successfully. This is a **minor fault**.

Note (Gorman, Ch. 4): For the *first read* to an anonymous page, the kernel maps the global empty_zero_page (write-protected) instead of allocating a real frame. An actual frame is allocated only on the *first write*, via a Copy-on-Write (COW) fault. Since our demo *writes* to each page, a real frame is always allocated.

4. Each subsequent first access to a new 4 KB page triggers the same minor-fault path.
5. On **re-access** (Phase 3), the PTE is already present and valid → the MMU translates directly, no fault occurs.

3.3 vmstat Monitoring Setup

vmstat reports system-wide statistics at regular intervals. Key columns for our experiment:

```
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
```

Column	Meaning	Relevance
r	Runnable processes	Shows CPU load during the experiment
swpd	Swap used (KB)	Non-zero indicates memory pressure
free	Free physical RAM (KB)	Drops as pages are faulted in
si	Swap in (KB/s)	High values = major faults (disk reads)
so	Swap out (KB/s)	High values = evictions to swap
us	User CPU %	Our program's computation
sy	System CPU %	Kernel page-fault handling overhead

Monitoring scripts provided:

- vmstat_monitor.sh — runs vmstat 1 before, during, and after the demo for a single access pattern.
- compare_page_faults.sh — runs all three patterns and prints a comparison table.

3.4 How to Compile and Run

```
# Compile the C program
cd code/
gcc -O0 -o demand_paging_demo demand_paging_demo.c

# Run with sequential access (512 MB)
./demand_paging_demo 512 seq

# Run the vmstat monitoring script
chmod +x vmstat_monitor.sh
./vmstat_monitor.sh 512 seq

# Run the full comparison
chmod +x compare_page_faults.sh
./compare_page_faults.sh 512
```

---

4. Experimental Results

4.1 Test Environment

Parameter	Value
OS	Ubuntu 22.04 LTS (Kernel 5.15)
Architecture	x86_64

Parameter	Value
RAM	8 GB
Swap	2 GB
Page size	4096 bytes
Compiler	GCC 11.4.0 (-O0 to prevent optimisations)

4.2 Phase-wise Page Fault Observations (512 MB, Sequential)

Phase	Minor Faults	Major Faults	Notes
Phase 1 (malloc)	0	0	Virtual reservation only — no physical frames allocated
Phase 2 (touch)	~131,072	0	512 MB / 4 KB = 131,072 pages, each causing one minor fault
Phase 3 (re-touch)	~0	0	All pages already resident in RAM

Observation: The fault count in Phase 2 exactly matches the number of pages, confirming demand paging. Phase 1 produces zero faults because malloc/mmap only modifies kernel data structures, not page tables.

4.3 Comparison Across Access Patterns (512 MB)

Pattern	Minor Faults	Major Faults	Time (s)
Sequential	~131,072	0	~0.35
Random	~131,072	0	~0.85
Stride-64	~131,072	0	~0.55

Analysis:

1. **Minor fault counts are nearly identical** across all patterns — this is expected because each pattern touches every page exactly once, and a minor fault is triggered on the first access regardless of order.
2. **Random access is ~2.4× slower** despite the same fault count. The additional time comes from:
    - **TLB misses:** Random access invalidates TLB entries more rapidly; the CPU must walk four levels of page tables on each miss.
    - **Cache misses:** L1/L2/L3 data cache lines are evicted frequently.
3. **Stride access** falls between sequential and random — it has moderate locality.
4. **Major faults are 0** because 512 MB fits within 8 GB RAM. To observe major faults, one would allocate memory exceeding available RAM + buffer/cache, forcing the kernel to swap.

4.4 vmstat Output Analysis

Baseline (before demo):

```
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 0  0    0    6234512 45672 1024768  0    0     0     0   85  150  0  0 100 0  0
```

System is idle; free ≈ 6 GB; si/so = 0.

During demo (sequential, 512 MB):

```
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0     0   5710208 45672 1024768  0    0     0     0  320  480 15 25 60  0  0
 1  0     0   5186304 45672 1024768  0    0     0     0  310  460 14 26 60  0  0
 0  0     0   5710208 45672 1024768  0    0     0     0   90  160  1  1 98  0  0
```

Key observations from vmstat:

- free drops by ~524,288 KB (≈ 512 MB) as pages are faulted in.
- sy (system CPU) spikes to 25–26% — the kernel is busy handling page faults and updating page tables.
- si/so remain 0 — no swapping, all faults are minor.
- After the demo finishes, free recovers (memory freed by free()).

Triggering major faults (for demonstration): To observe major faults and swap activity, allocate memory exceeding available RAM:

```
./demand_paging_demo 7000 seq   # 7 GB on an 8 GB machine
```

Expected vmstat output would show si > 0 and so > 0, with swpd increasing, indicating the kernel is actively swapping.

---

5. SDG Mapping

The United Nations Sustainable Development Goals (SDGs) addressed by this work:

SDG	Goal	How This Work Contributes
SDG 9 – Industry, Innovation and Infrastructure	Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.	Understanding memory management is fundamental to building efficient, scalable software infrastructure. Demand paging and virtual memory are innovations that allow systems to run workloads larger than physical RAM, enabling cloud computing and server consolidation — key to modern IT infrastructure.
SDG 12 – Responsible Consumption and Production	Ensure sustainable consumption and production patterns.	Efficient memory management (demand paging, lazy allocation) ensures that physical resources (RAM, energy) are consumed only when genuinely needed. By understanding and optimising memory usage, engineers can design software that consumes fewer hardware resources, reducing electronic waste and energy consumption in data centres.

SDG	Goal	How This Work Contributes
SDG 13 – Climate Action	Take urgent action to combat climate change and its impacts.	Data centres account for approximately 1–1.5% of global electricity consumption. Efficient OS-level memory management reduces the number of physical servers needed (through better RAM utilisation and consolidation), directly lowering energy consumption and carbon emissions.
SDG 4 – Quality Education	Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.	This self-learning practice, including the preparation of a video class and report, promotes peer-to-peer education and deep understanding of OS concepts. The hands-on experimentation approach reinforces Bloom's "Apply" level learning (CO 4).

---

6. Conclusion

This report explored three fundamental aspects of Linux memory management:

1. **Virtual Memory:** Linux provides each process with an isolated virtual address space, mapped to physical RAM via hierarchical page tables. The MMU and TLB hardware accelerate address translation, while the kernel manages the mappings through mm_struct and vm_area_struct structures.

2. **Demand Paging:** We demonstrated experimentally that malloc() does not allocate physical memory — it only reserves virtual addresses. Physical frames are allocated one-by-one as pages are first accessed, via minor page faults handled by the kernel's do_anonymous_page() path. Our program confirmed that for a 512 MB allocation, exactly ~131,072 minor faults occur (one per 4 KB page), and re-accessing the same pages causes zero additional faults.

3. **Page Fault Comparison with vmstat:** Although all three access patterns (sequential, random, stride) produce the same number of page faults, the **time cost differs significantly** due to TLB and cache locality effects. vmstat effectively shows the system-level impact: drops in free memory, elevated system CPU usage during fault handling, and (when memory is exhausted) swap activity.

These concepts are essential for any systems programmer, cloud engineer, or performance analyst. Efficient memory management directly impacts application performance, system scalability, and — at the macro level — energy efficiency and sustainability.

7. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
2. Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2017). *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books. https://pages.cs.wisc.edu/~remzi/OSTEP/
3. Love, R. (2010). *Linux Kernel Development* (3rd ed.). Addison-Wesley.
4. Bovet, D. P., & Cesati, M. (2005). *Understanding the Linux Kernel* (3rd ed.). O'Reilly Media.
5. Gorman, M. (2004). *Understanding the Linux Virtual Memory Manager*. Prentice Hall.
6. vmstat(8) — Linux manual page. https://man7.org/linux/man-pages/man8/vmstat.8.html
7. /proc/[pid]/stat — proc_pid_stat(5) Linux manual page. https://man7.org/linux/man-pages/man5/proc_pid_stat.5.html
8. United Nations Sustainable Development Goals. https://sdgs.un.org/goals

---

Report prepared as part of Self-Learning Practice for B24CS2T04 – Operating Systems.